

Laboratorio integrador unidad 01

1. Propósito del laboratorio

En este laboratorio cada equipo configurará un entorno GNU/Linux en una máquina virtual y desarrollará una aplicación Python para procesar archivos de texto. El sistema deberá ejecutar tareas concurrentemente, usar recursos compartidos de manera controlada, generar reportes, administrar archivos y permitir observar el consumo de memoria y los procesos del sistema operativo, este laboratorio conecta los siguientes contenidos:

- Procesos, PID, PPID, estados y cambio de contexto.
- Hilos, concurrencia, paralelismo, recursos compartidos y secciones críticas.
- Administración de memoria y consumo de recursos por procesos.
- Sistema de archivos: directorios, permisos, lectura, escritura, movimiento de archivos y bitácoras.

2. Resultados de aprendizaje

Al terminar el laboratorio, el equipo será capaz de:

1. Instalar y utilizar una distribución GNU/Linux dentro de una máquina virtual.
2. Identificar procesos mediante PID, PPID, estado, uso de CPU y consumo de memoria.
3. Implementar una solución secuencial y otra concurrente en Python.
4. Crear y coordinar hilos para procesar tareas distribuidas mediante una cola de trabajo.
5. Identificar una condición de carrera y corregirla con un mecanismo de sincronización.
6. Aplicar operaciones de administración de archivos y directorios desde Python y GNU/Linux.
7. Medir tiempos de ejecución y uso de memoria, interpretando los resultados obtenidos.
8. Comunicar decisiones, resultados y evidencias en un informe técnico.

3. Software y distribución

3.1 Herramientas obligatorias

Herramienta	Uso en el laboratorio
Oracle VM VirtualBox	Ejecución de GNU/Linux en una máquina virtual
Ubuntu Desktop 24.04 LTS, 64 bits	Sistema operativo invitado
Python 3	Desarrollo de la aplicación concurrente
Terminal de GNU/Linux	Observación de procesos, memoria y sistema de archivos
Editor de código	VS Code, nano, vim, PyCharm Community u otro equivalente

3.2 Distribución seleccionada: Ubuntu Desktop 24.04 LTS

Se utilizará Ubuntu Desktop 24.04 LTS de 64 bits, esta distribución se selecciona porque:

- Posee soporte de largo plazo, lo que reduce cambios inesperados de paquetes y versiones durante el semestre.
- Está ampliamente documentada y cuenta con una comunidad extensa, facilitando la resolución de problemas de instalación y configuración.
- Incluye una interfaz gráfica apropiada para estudiantes que se inician en GNU/Linux, sin impedir el trabajo desde terminal.
- Dispone de repositorios oficiales y de herramientas estándar para observar procesos, memoria y sistemas de archivos, tales como ps, top, free, df, du, pstree y vmstat.
- Python 3 y herramientas de desarrollo se instalan directamente desde sus repositorios.

La imagen oficial de Ubuntu Desktop 24.04.4 LTS está disponible para arquitectura Intel/AMD de 64 bits; la documentación de lanzamiento indica que se requieren al menos 1024 MiB de RAM para instalar desde la imagen, para una experiencia adecuada dentro de una máquina virtual, este laboratorio configurará más memoria y almacenamiento.

3.3 Requisitos del computador anfitrión

Recurso	Mínimo recomendado	Recomendado
RAM total del computador	8 GB	16 GB o más
Espacio libre en disco	35 GB	50 GB o más
Procesador	64 bits con virtualización habilitada	4 núcleos o más
Conectividad	Acceso a Internet	Acceso estable durante la instalación

Si VirtualBox informa que no puede iniciar máquinas de 64 bits, el equipo debe revisar que la virtualización por hardware esté habilitada en BIOS/UEFI y que no exista conflicto con otros hipervisores del sistema anfitrión (si debe realizarlo, investigue como hacerlo)

4. Instalación de VirtualBox

4.1 Obtención segura

1. Ingresar al sitio oficial de descarga de VirtualBox: [VirtualBox Downloads](#).
2. Descargar el instalador correspondiente al sistema operativo anfitrión:
 - Windows hosts para computadores con Windows.
 - macOS / OS X hosts para computadores Mac compatibles.
3. Ejecutar el instalador descargado.
4. Mantener las opciones predeterminadas, excepto que el sistema solicite permisos de administrador.
5. Aceptar la instalación de controladores de red de Oracle si el sistema lo solicita.
6. Abrir VirtualBox y verificar que la aplicación se ejecute correctamente.

El manual oficial de VirtualBox indica que la instalación depende del sistema operativo anfitrión y que se distribuye en paquetes específicos para cada plataforma.

4.2 Verificación inicial

Antes de continuar, cada integrante responsable del entorno debe comprobar:

- VirtualBox abre sin mensajes críticos.
- El computador dispone de espacio libre suficiente.
- La virtualización está habilitada.
- Se puede crear una máquina virtual de tipo Linux de 64 bits.

5. Instalación y configuración de Ubuntu

5.1 Descargar la imagen ISO

1. Ingresar al sitio oficial de Ubuntu: [Ubuntu Desktop](#).
2. Descargar la imagen ISO de Ubuntu Desktop 24.04 LTS, 64 bits.
3. Guardar la ISO en una carpeta conocida, por ejemplo, Descargas.
4. No es necesario crear un pendrive booteable: la ISO se conectará como unidad óptica virtual en VirtualBox.

Ubuntu entrega imágenes oficiales de escritorio para probar o instalar el sistema sin modificar inicialmente el sistema operativo anfitrión.

5.2 Crear la máquina virtual

En VirtualBox, seleccionar Nueva y utilizar la siguiente configuración:

Parámetro	Configuración solicitada
Nombre	Ubuntu-SO-EquipoXX
Tipo	Linux
Versión	Ubuntu (64-bit)
Imagen ISO	Archivo ISO descargado de Ubuntu 24.04 LTS
Memoria RAM	4096 MB como mínimo; 6144 MB si el anfitrión posee 16 GB o más
Procesadores	2; usar 4 si el computador anfitrión tiene capacidad suficiente

Disco virtual	VDI, reservado dinámicamente
Tamaño del disco	30 GB como mínimo

5.3 Instalar Ubuntu

1. Iniciar la máquina virtual creada.
2. En la pantalla de bienvenida, seleccionar idioma y distribución de teclado.
3. Elegir Install Ubuntu.
4. Seleccionar instalación interactiva y mantener la configuración predeterminada cuando no exista una indicación distinta.
5. Para el disco, seleccionar la opción que instala Ubuntu sobre el disco virtual. Esta opción afecta únicamente el archivo de disco de la máquina virtual, no el disco físico principal del computador anfitrión.
6. Definir zona horaria.
7. Crear una cuenta de usuario. Se recomienda:
 - Nombre visible: Equipo XX.
 - Nombre de usuario: equipoXX.
 - Contraseña: definida y resguardada por el equipo.
8. Esperar la finalización de la instalación y reiniciar la máquina virtual cuando se solicite.
9. Si se solicita retirar el medio de instalación, presionar Enter; VirtualBox desconectará o ignorará la ISO según la configuración.

5.4 Configuración posterior

Abrir una terminal y ejecutar los siguientes comandos:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv git htop tree
python3 --version
```

Crear la carpeta de trabajo:

```
mkdir -p ~/laboratorio_so/{src,data/entrada,data/procesados,data/errores,data/reportes,logs,evidencia}  
cd ~/laboratorio_so  
pwd  
tree
```

Verificar el sistema y los recursos disponibles:

```
uname -a  
whoami  
free -h  
df -h  
python3 --version
```

6. Desafío integrador

6.1 Contexto

Una organización recibe archivos de texto en una carpeta de entrada. Cada archivo representa un registro que debe ser procesado. El sistema debe leer los archivos, calcular métricas, generar reportes, mover los archivos ya tratados y registrar todos los eventos relevantes.

6.2 Problema a resolver

Cada equipo debe implementar el sistema Procesador Concurrente de Archivos, cumpliendo los siguientes requisitos, requisitos funcionales

1. Detectar archivos .txt dentro de data/entrada/.
2. Leer el contenido de cada archivo.
3. Calcular por archivo:
 - Cantidad de líneas.
 - Cantidad de palabras.
 - Cantidad de caracteres.

- Palabra más frecuente, ignorando mayúsculas/minúsculas.
- 4. Crear un reporte individual en data/reportes/.
- 5. Mover el archivo procesado a data/procesados/.
- 6. Registrar cada evento en logs/sistema.log.
- 7. Registrar los errores en data/errores/ o en una sección diferenciada del log.
- 8. Generar un reporte consolidado con total de archivos, palabras y caracteres procesados.

Requisitos de procesos e hilos

1. Ejecutar la solución y observar el proceso Python desde GNU/Linux.
2. Registrar PID, PPID, estado y consumo de CPU/memoria del programa.
3. Implementar una versión secuencial que procese los archivos uno a uno.
4. Implementar una versión concurrente con:
 - Un hilo productor que agrega rutas de archivos a una cola.
 - Al menos tres hilos trabajadores que consumen y procesan archivos.
 - Una cola compartida mediante queue.Queue.
 - Un mecanismo para esperar el término ordenado de los trabajadores.
5. Demostrar una condición de carrera y corregirla mediante threading.Lock u otro mecanismo justificado.

El módulo queue de Python implementa colas sincronizadas para escenarios con múltiples productores y consumidores, por lo que resulta adecuado para distribuir archivos entre hilos de forma segura.

Requisitos de memoria

1. Registrar el consumo de memoria antes de ejecutar el programa.
2. Registrar el consumo de memoria durante la versión secuencial.
3. Registrar el consumo de memoria durante la versión concurrente.

4. Repetir las pruebas con al menos dos volúmenes de trabajo diferentes.
5. Comparar e interpretar RSS, VSZ y el porcentaje de memoria observado.

Requisitos del sistema de archivos

1. Crear y usar las carpetas indicadas.
2. Leer y escribir archivos de texto correctamente.
3. Mover archivos procesados sin duplicarlos accidentalmente.
4. Registrar resultados y errores en archivos persistentes.
5. Mostrar permisos y tamaño de archivos mediante comandos GNU/Linux.

7. Ejemplo guiado replicable

Este ejemplo es una versión reducida del desafío, el objetivo es que el equipo pueda ejecutarlo, observarlo y luego extenderlo durante el laboratorio.

7.1 Preparación de archivos de entrada

Ubicados en `~/laboratorio_so`, ejecutar:

```
cd ~/laboratorio_so
printf 'hola mundo hola sistemas operativos\n' > data/entrada/archivo1.txt
printf 'procesos hilos memoria archivos procesos\n' > data/entrada/archivo2.txt
printf 'concurrency concurrency seccion critica\n' > data/entrada/archivo3.txt
ls -lah data/entrada
```

7.2 Crear el programa de ejemplo

Crear el archivo `src/ejemplo_guiado.py` con el siguiente contenido:


```

1  from pathlib import Path
2  from collections import Counter
3  from queue import Queue
4  from threading import Thread, Lock
5  from datetime import datetime
6  import shutil
7
8  BASE = Path.home() / "laboratorio_so"
9  ENTRADA = BASE / "data" / "entrada"
10 PROCESADOS = BASE / "data" / "procesados"
11 REPORTEES = BASE / "data" / "reportes"
12 LOG = BASE / "logs" / "sistema.log"
13 cola = Queue()
14 bloqueo_log = Lock()
15 bloqueo_totales = Lock()
16 totales = {"archivos": 0, "palabras": 0, "caracteres": 0}
17 def registrar(mensaje):
18     marca_tiempo = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
19     with bloqueo_log:
20         with LOG.open("a", encoding="utf-8") as archivo_log:
21             archivo_log.write(f"[{marca_tiempo}] {mensaje}\n")

```

```

22 def procesar_archivo(ruta):
23     contenido = ruta.read_text(encoding="utf-8")
24     palabras = contenido.lower().split()
25     frecuencia = Counter(palabras)
26     palabra_frecuente = frecuencia.most_common(1)[0][0] if palabras else "Sin palabras"
27     reporte = REPORTEES / f"reporte_{ruta.stem}.txt"
28     reporte.write_text(
29         f"Archivo: {ruta.name}\n"
30         f"Líneas: {len(contenido.splitlines())}\n"
31         f"Palabras: {len(palabras)}\n"
32         f"Caracteres: {len(contenido)}\n"
33         f"Palabra más frecuente: {palabra_frecuente}\n",
34         encoding="utf-8",
35     )
36     with bloqueo_totales:
37         totales["archivos"] += 1
38         totales["palabras"] += len(palabras)
39         totales["caracteres"] += len(contenido)
40     shutil.move(str(ruta), PROCESADOS / ruta.name)
41     registrar(f"Procesado correctamente: {ruta.name}")

```

```

42 def productor():
43     for ruta in ENTRADA.glob("*.txt"):
44         cola.put(ruta)
45         registrar(f"Archivo agregado a la cola: {ruta.name}")
46 def trabajador(numero):
47     while True:
48         ruta = cola.get()
49         if ruta is None:
50             cola.task_done()
51             break
52         try:
53             procesar_archivo(ruta)
54             print(f"Trabajador {numero}: procesó {ruta.name}")
55         except Exception as error:
56             registrar(f"ERROR en {ruta.name}: {error}")
57         finally:
58             cola.task_done()

```

```

59 def main():
60     for carpeta in [ENTRADA, PROCESADOS, REPORTES, LOG.parent]:
61         carpeta.mkdir(parents=True, exist_ok=True)
62     cantidad_trabajadores = 3
63     trabajadores = [Thread(target=trabajador, args=(i + 1,)) for i in range(cantidad_trabajadores)]
64     for hilo in trabajadores:
65         hilo.start()
66     hilo_productor = Thread(target=productor)
67     hilo_productor.start()
68     hilo_productor.join()
69     for _ in trabajadores:
70         cola.put(None)
71     cola.join()
72     for hilo in trabajadores:
73         hilo.join()
74     consolidado = REPORTES / "reporte_consolidado.txt"
75     consolidado.write_text(
76         f"Archivos procesados: {totales['archivos']}\n"
77         f"Palabras procesadas: {totales['palabras']}\n"
78         f"Caracteres procesados: {totales['caracteres']}\n",
79         encoding="utf-8",
80     )

```

```

81     registrar("Ejecución finalizada correctamente")
82     print("Proceso terminado. Revise data/reportes y logs/sistema.log")
83 if __name__ == "__main__":
84     main()

```

7.3 Ejecutar el ejemplo

```
cd ~/laboratorio_so
python3 src/ejemplo_guiado.py
find data -type f -printf '%p\n'
cat data/reportes/reporte_consolidado.txt
cat logs/sistema.log
```

Resultado esperado:

- Los archivos dejan de estar en data/entrada/.
- Los archivos aparecen en data/procesados/.
- Se crea un reporte por archivo en data/reportes/.
- Se crea un reporte consolidado.
- El log registra eventos de entrada, procesamiento y finalización.

7.4 Observar el proceso desde otra terminal

Para que el proceso sea observable durante más tiempo, agregue temporalmente al inicio de `procesar_archivo`:

```
import time
time.sleep(3)
```

En una terminal, ejecutar el programa:

```
python3 src/ejemplo_guiado.py
```

En otra terminal, mientras está ejecutándose:

```
ps -ef | grep ejemplo_guiado
ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,cmd -C python3
pstree -p
htop
```

Registrar una captura donde se observe el proceso Python e identificar:

- PID: identificador del proceso.
- PPID: identificador del proceso padre, normalmente la terminal que lo inició.
- STAT: estado del proceso.
- %CPU: porcentaje de uso de CPU.
- %MEM, RSS y VSZ: indicadores de consumo de memoria.

7.5 Experimento de condición de carrera

Para demostrar un problema de concurrencia, realizar una copia del programa denominada `src/ejemplo_sin_lock.py` y efectuar temporalmente estos cambios:

1. Eliminar `bloqueo_totales = Lock()`.
2. Reemplazar el bloque protegido por:

```
totales["archivos"] += 1 # type: ignore
totales["palabras"] += len(palabras) # type: ignore
totales["caracteres"] += len(contenido) # type: ignore
```

3. Crear al menos 100 archivos de entrada, por ejemplo:

```
for i in $(seq 1 100); do
| printf 'uno dos tres cuatro cinco procesos hilos memoria\n' > "data/entrada/archivo_${i}.txt"
```

4. Ejecutar la versión sin bloqueo varias veces y comparar el total esperado con el total registrado.
5. Restaurar el uso de `Lock` y ejecutar nuevamente.

El informe debe explicar que el contador compartido y el archivo de log son recursos compartidos; una sección crítica requiere exclusión mutua para evitar actualizaciones inconsistentes. El uso de `with bloqueo_totales` garantiza que un solo hilo actualice los totales protegidos a la vez.

Nota docente: debido a particularidades de planificación y del intérprete, una condición de carrera puede no manifestarse en todas las ejecuciones. Para hacer el efecto más visible, los equipos pueden separar la lectura y escritura del contador e introducir una pausa breve entre ambas operaciones únicamente en la versión experimental.

Ejemplo experimental deliberadamente inseguro:

```
import time
valor_actual = totales["archivos"] # type: ignore
time.sleep(0.001)
totales["archivos"] = valor_actual + 1 # type: ignore
```

7.6 Medición de memoria y disco

Antes, durante y después de cada ejecución, registrar:

```
free -h # pyright: ignore[reportUndefinedVariable]
df -h # pyright: ignore[reportUndefinedVariable]
ps -o pid,ppid,stat,%cpu,%mem,rss,vsz,cmd -C python3 # pyright: ignore[reportUndefinedVariable]
```

También se recomienda medir el tiempo de ejecución:

```
time python3 src/ejemplo_guiado.py
```

Para comparar secuencialidad y concurrencia, los equipos deben crear una versión que procese los archivos en un ciclo simple, sin Thread, Queue ni Lock, y medirla con la misma entrada.

9. Entregables

Cada equipo debe entregar una carpeta comprimida o repositorio con la siguiente estructura mínima:

```
laboratorio_so_equipoXX/
├── README.md
├── src/
│   ├── version_secuencial.py
│   ├── version_concurrente.py
│   └── version_sincronizacion_experimental.py
├── data/
│   ├── entrada/                # Puede contener ejemplos pequeños
│   ├── procesados/
│   ├── errores/
│   └── reportes/
├── logs/
│   └── sistema.log
├── evidencia/
│   ├── capturas/
│   ├── comandos_ejecutados.txt
│   └── mediciones.csv o mediciones.md
├── informe.pdf
└── requirements.txt
```

9.1 README obligatorio

El archivo README.md debe indicar:

- Integrantes y roles.
- Requisitos para ejecutar el proyecto.
- Instrucciones paso a paso de ejecución.
- Comandos utilizados para ejecutar cada versión.
- Estructura de carpetas.
- Mecanismo de concurrencia aplicado.
- Cómo reproducir el experimento de condición de carrera.

9.2 Informe técnico

Extensión: mínimo 8 y máximo 12 páginas, sin considerar portada, índice, anexos, capturas suplementarias ni código fuente extenso.

Estructura requerida:

1. Portada.
2. Introducción y objetivos.
3. Ambiente de ejecución: máquina virtual, distribución, recursos asignados y versiones.
4. Diseño de la solución: diagrama de componentes, flujo de archivos, procesos, hilos y recursos compartidos.
5. Implementación: versión secuencial, versión concurrente y manejo de errores.
6. Experimento de sincronización: condición de carrera, evidencia, corrección y explicación.
7. Medición y análisis: tiempo, memoria, uso de CPU y sistema de archivos.
8. Conclusiones: mínimo tres, directamente vinculadas con los resultados obtenidos.
9. Anexos: código relevante, capturas, comandos y registros adicionales.

10. Rúbrica de evaluación

Puntaje total: 100 puntos

Criterio	Nivel destacado	Nivel adecuado	Nivel básico	Nivel insuficiente	Puntaje máximo
Instalación y configuración del ambiente	VirtualBox y Ubuntu operativos; configuración documentada, recursos	Ambiente operativo y documentado, con pequeñas omisiones de detalle	Ambiente parcialmente operativo o documentación incompleta	No hay evidencia funcional del ambiente GNU/Linux	15

	justificados y herramientas instaladas correctamente				
Evidencia de procesos	Identifica e interpreta correctamente PID, PPID, estado, CPU y memoria; presenta evidencia clara con comandos	Identifica procesos y presenta evidencia suficiente, con análisis parcial	Presenta capturas o comandos, pero confunde conceptos o no interpreta resultados	No evidencia ejecución ni observación de procesos	10
Versión secuencial	Procesa correctamente todos los archivos, genera reportes, logs y maneja errores básicos	Cumple la mayor parte de las funcionalidades con errores menores	Procesa solo parcialmente o presenta resultados incompletos	No existe o no ejecuta	10
Concurrencia con hilos	Implementa productor, cola segura, tres o más trabajadores,	Usa hilos y cola de forma funcional, con limitaciones menores	Usa hilos, pero sin coordinación correcta o con	No implementa concurrencia funcional	20

	finalización ordenada y código modular		conurrencia limitada		
Sincronización y sección crítica	Demuestra condición de carrera, entrega evidencia, aplica mecanismo apropiado y justifica técnicamente la solución	Identifica recurso compartido y aplica una protección funcional, con evidencia parcial	Usa un mecanismo de sincronización sin demostrar claramente el problema ni explicar su efecto	No reconoce o no soluciona accesos concurrentes conflictivos	15
Administración de memoria y rendimiento	Presenta mediciones comparables de ambas versiones, interpreta RSS/VSZ, memoria y tiempos con conclusiones razonadas	Registra y compara mediciones, con interpretación parcial	Incluye datos aislados sin comparación o análisis insuficiente	No mide ni analiza memoria o rendimiento	10
Sistema de archivos	Maneja correctamente e directorios,	Cumple la mayoría de las	Maneja archivos de	No hay manejo funcional de archivos	10

		lectura, escritura, movimiento, reportes, logs y errores	operaciones solicitadas	manera parcial o frágil		
Calidad de código y README	Código modular, legible, ejecutable, con nombres claros, manejo básico de errores y README reproducible	Código funcional y ordenado, con documentación suficiente	Código difícil de ejecutar, duplicado o con instrucciones incompletas	Código incompleto, no ejecutable o sin documentación	5	
Informe técnico	Documento claro, estructurado, con evidencias pertinentes, análisis técnico y conclusiones basadas en datos	Informe completo con análisis suficiente y pequeñas omisiones	Informe mayormente descriptivo, con poca evidencia o conclusiones débiles	Informe incompleto, sin análisis o sin evidencias	5	

11. Preguntas orientadoras para el análisis

El informe debe responder, con evidencia obtenida durante el laboratorio, al menos las siguientes preguntas:

1. ¿Qué PID, PPID y estado presentó el proceso Python durante su ejecución?
2. ¿Qué recursos pertenecen al proceso principal de la aplicación?
3. ¿Qué recursos comparten los hilos trabajadores?
4. ¿Por qué una cola sincronizada permite distribuir trabajo entre consumidores?
5. ¿Qué sección crítica identificó el equipo y qué resultado incorrecto puede ocurrir sin sincronización?
6. ¿Cómo cambia el tiempo total al usar varios hilos? ¿El resultado observado se explica por CPU, E/S de archivos, coordinación u otro factor?
7. ¿Qué diferencias se observaron entre la versión secuencial y la concurrente en consumo de memoria?
8. ¿Qué significan RSS y VSZ en la salida de ps?
9. ¿Qué riesgos existen cuando varios hilos escriben sobre el mismo archivo de log o actualizan el mismo contador?
10. ¿Qué limitaciones tuvo la solución y cómo la mejorarían en una versión futura?

12. Recomendaciones de trabajo colaborativo

- Definir roles, sin impedir que todos participen en todas las etapas: responsable de ambiente, responsable de concurrencia, responsable de pruebas/evidencias y responsable de integración/documentación.
- Usar Git cuando sea posible y realizar commits con mensajes significativos.
- Probar primero con pocos archivos y luego aumentar gradualmente la carga.
- No eliminar evidencias de ejecución: guardar capturas, salidas de terminal y archivos de log.

- No confundir concurrencia con paralelismo: varios hilos pueden avanzar de forma intercalada, pero el paralelismo efectivo depende de los núcleos disponibles y del tipo de tarea.
- Explicar decisiones técnicas usando resultados de las pruebas, no solo definiciones teóricas.